Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen III: Monografía Completa de Rust – Seguridad de Memoria, Concurrencia y Sistemas

Parte V (Bloque A–H): De cero a sistemas de alto rendimiento, programación asíncrona, unsafe y arquitectura

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Bloques A–C: Fundamentos, Entorno y Nivel Principiante en Rust 1

1. A. Introducción, historia, filosofía y modelo de compilación de Rust 2
1.1. Historia, problema que resuelve y filosofía de diseño 2
1.2. Modelo de ejecución, fases HIR/MIR/LLVM y comparativa con otros lenguajes 2

2. B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo 5
2.1. Bloque B: Toolchain oficial (rustup, rustc, cargo, clippy, rustfmt) y estructura de un crate 5
2.2. Bloque C: Variables, tipos primitivos, expresiones de control de flujo y funciones 5

II Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad 8

3. D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes) 9
3.1. Las tres leyes de Ownership, semántica Move frente a Copy y el rasgo Drop 9
3.2. Reglas de Borrowing (&T vs &mut T), Non-Lexical Lifetimes (NLL) y parámetros de vida 'a 10

4. D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result) 12
4.1. Estructuras, enumeraciones algebraicas, optimización de nicho y coincidencia de patrones 12
4.2. Rasgos (Traits), genéricos, tipos asociados y manejo de errores con Option<T> y Result<T, E> 12

III Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe 15

5. E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior 16
5.1. Clausuras (Fn, FnMut, FnOnce) e iteradores perezosos de coste cero 16
5.2. Punteros inteligentes (Box, Rc, Weak) y mutabilidad interior (Cell, RefCell, Cow) 16

6. E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio) 19
6.1. Concurrencia basada en hilos: Send, Sync, thread::scope, Arc, Mutex y canales 19
6.2. Programación asíncrona: el rasgo Future, máquinas de estados, Pin/Unpin y el ejecutor Tokio 20

7. F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización 22
7.1. Rust inseguro (unsafe), contratos de seguridad, Miri e interoperabilidad FFI con C 22
7.2. Metaprogramación con macros, entornos empotrados (#![no_std]), WebAssembly y optimización SIMD 23

IV Bloques G−H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos **25**

8. G−H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust **26**
8.1. Bloque G: Ecosistema de crates, Cargo Workspaces, idioms de diseño de APIs y antipatrones 26
8.2. Bloque H: Evaluación integral y los 4 proyectos progresivos (incluyendo rust-kv-engine verificado) 26

A. Notación **29**

B. Hoja de fórmulas **30**

C. Soluciones **31**
C.1. A. Introducción, historia, filosofía y modelo de compilación de Rust 31
C.2. B−C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo 31
C.3. D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes) 32
C.4. D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result) 33
C.5. E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior 33
C.6. E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio) 34
C.7. F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización 35
C.8. G−H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust . . 35

D. Auditoría de fuentes **37**

E. Plan de estudio **39**

Parte I

Bloques A–C: Fundamentos, Entorno y Nivel Principiante en Rust

Capítulo 1

A. Introducción, historia, filosofía y modelo de compilación de Rust

Este capítulo inaugura la monografía autónoma de Rust (Bloque A). Un lector sin experiencia previa en C o C++ comprenderá qué problema fundamental de la ingeniería de sistemas resuelve Rust: garantizar simultáneamente control predictivo sobre la representación de memoria y ausencia matemática de comportamiento indefinido (Undefined Behavior) en código seguro (`safe Rust`), sin recurrir a un recolector de basura en tiempo de ejecución.



Figura 1. Tubería del compilador rustc: análisis sintáctico, expansión de macros, HIR, inferencia de tipos, MIR (borrow checker NLL) y optimización LLVM.

Consejo

No luches contra el comprobador de préstamos intentando replicar grafos cíclicos de punteros mutables propios de Java o C++; modela la propiedad de los datos como árboles jerárquicos o utiliza índices tipados sobre vectores contiguos (arenas).

Definición

Seguridad espacial y temporal de memoria en Rust: propiedad verificada estáticamente en MIR según la cual ningún acceso seguro lee o escribe fuera de los límites asignados (seguridad espacial) ni accede a una región cuya vida útil ha concluido o está siendo mutada concurrentemente (seguridad temporal).

Autoficha

¿Por qué un puntero a objeto de rasgo `&dyn Write` ocupa 16 bytes en una arquitectura de 64 bits mientras que una referencia ordinaria `&File` ocupa únicamente 8 bytes?

1.1 Historia, problema que resuelve y filosofía de diseño

Iniciado en 2006 por Graydon Hoare y respaldado por Mozilla Research desde 2009 para construir el motor experimental Servo, Rust alcanzó su versión estable 1.0 en mayo de 2015 bajo un compromiso estricto de estabilidad sin estancamiento mediante el sistema de Ediciones (2015, 2018, 2021 y 2024). Las estadísticas de vulnerabilidades publicadas por Microsoft Security Response Center y el proyecto Chromium revelaron que aproximadamente el 70 % de las vulnerabilidades críticas de seguridad en grandes bases de código C y C++ proceden de defectos de seguridad de memoria: uso después de liberación (use-after-free), doble liberación (double free), desbordamientos de búfer, desreferenciación de punteros colgantes (dangling pointers) y carreras de datos (data races) entre hilos. Como se ilustra en la Figura \ref{fig:fig-rust-compiler-pipeline}, Rust traslada la verificación de estas invariantes desde el tiempo de ejecución hacia la representación intermedia de nivel medio (MIR, Mid-level Intermediate Representation) durante la compilación.

1.2 Modelo de ejecución, fases HIR/MIR/LLVM y comparativa con otros lenguajes

El principio de abstracciones de coste cero (zero-cost abstractions), formulado originalmente por Bjarne Stroustrup, establece que aquello que no se utiliza no conlleva coste alguno y aquello que sí se utiliza se

compila con una eficiencia igual o superior al código escrito manualmente a bajo nivel. En Rust, los tipos genéricos parametrizados $f\langle T : \text{Trait}\rangle(x : T)$ se compilan por defecto mediante monomorfización estática: el compilador genera una copia especializada de la función para cada tipo concreto invocado, eliminando la indirección de tablas virtuales (`vtable`) y permitiendo el *inlining* agresivo del optimizador LLVM:

```
// Compilado mediante monomorfización a instrucciones máquina directas sin vtable:
#[inline]
pub fn clamp_value<T: PartialOrd>(val: T, low: T, high: T) -> T {
    if val < low { low } else if val > high { high } else { val }
}
```

Cuando el despacho dinámico es deseable para reducir el tamaño del binario o almacenar colecciones heterogéneas, el programador lo solicita explícitamente mediante objetos de rasgo (`&dyn Trait` o `Box<dyn Trait>`), representados como un puntero ancho (*fat pointer*) de dos palabras máquina ($2 \times 8 = 16$ B en arquitecturas de 64 bits): una palabra para la dirección del dato y otra para la tabla virtual del rasgo.

Problema resuelto

Enunciado En una arquitectura de 64 bits ($W = 8$ B), un vector en Rust `Vec<T>` se representa en la pila mediante tres campos de tamaño palabra (`ptr`, `len`, `cap`), mientras que una rebanada `&[T]` es un puntero ancho de dos palabras (`ptr`, `len`). Calcula el tamaño total en pila en bytes de una estructura que contiene dos `Vec<u8>` y tres rebanadas `&[u8]`.

Datos

- $W = 8\,\text{B}$ — Tamaño de palabra en arquitectura de 64 bits
- $n_{vec} = 2$ — Número de vectores Vec<u8>
- $n_{slice} = 3$ — Número de rebanadas &[u8]

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Representación en pila de Vec<T> (3 palabras) y &[T] (2 palabras)

$$S = 2 \cdot (3 \cdot 8) + 3 \cdot (2 \cdot 8)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Tamaño de cada Vec<u8> en pila (3 * 8 = 24 B) y cada &[u8] (2 * 8 = 16 B)

$$S = 2 \cdot (3 \cdot 8) + 3 \cdot (2 \cdot 8)$$

$$((2 \cdot (3 \cdot 8)) + (3 \cdot (2 \cdot 8)))$$

Resultado

$$\textit{Resultado} = 96. \textit{La estructura ocupa exactamente 96 bytes en la pila (48 B de los dos vectores y 48 B de las tres rebanadas), indep}$$

Interpretación La estructura ocupa exactamente 96 bytes en la pila (48 B de los dos vectores y 48 B de las tres rebanadas), independientemente del número de elementos alojados en el montículo.

Comprobación de los pasos

- Paso 1: 96 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0004] (Básico, Numérico) Calcula el tamaño en bytes en la pila de un `Vec<f64>` en una arquitectura de 64 bits (3×8).
2 [PRB-0005] (Básico, Numérico) Calcula el tamaño en bytes de un puntero ancho `&dyn std::fmt::Display` en una CPU de 64 bits (2×8).
3 [PRB-0006] (Intermedio, Desarrollo por pasos) Si un binario contiene 45 funciones genéricas instanciadas cada una para 4 tipos concretos distintos mediante monomorfización, ¿cuántas funciones especializadas se emiten antes de la eliminación de código muerto?
4 [PRB-0007] (Intermedio, Diseño) Un búfer de 4096 elementos `u32` (4 bytes por elemento) está referenciado por un `Vec<u32>` (24 bytes en pila). ¿Cuántos bytes totales suma la cabecera en pila más la región en el montículo?

Capítulo 2

B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

Este capítulo cubre los Bloques B y C de la plantilla estandarizada: desde la configuración profesional del entorno (`rustup`, `cargo`, `Cargo.toml`, `Cargo.lock`, `rustfmt`, `clippy`, `rust-analyzer`) hasta el dominio completo del nivel principiante de Rust. A diferencia de los lenguajes imperativos clásicos donde casi todas las construcciones son sentencias sin valor, Rust es un lenguaje orientado a expresiones donde `if`, `match` y los bloques `{ ... }` producen valores tipados verificados estáticamente.

Consejo

Prefiere siempre `checked_add`, `saturating_add` o `wrapping_add` cuando implementes protocolos binarios, criptografía o contadores de sistemas para que el comportamiento ante desbordamiento no dependa del perfil de compilación.

Definición

Expresión de bloque en Rust: secuencia de sentencias encerrada entre llaves `{ s_1; s_2; ...; e }` cuyo tipo y valor resultante coinciden con los de la expresión final `e` (sin punto y coma), o con el tipo unidad `()` si termina en punto y coma.

Autoficha

¿Qué diferencia fundamental existe entre sombrear una variable (`let x = x + 1;`) y declararla mutable (`let mut x = 0; x += 1;`) en cuanto a la posibilidad de cambiar su tipo estático?

2.1 Bloque B: Toolchain oficial (rustup, rustc, cargo, clippy, rustfmt) y estructura de un crate

La herramienta `rustup` administra canales de distribución (`stable`, `beta`, `nightly`) y objetivos de compilación cruzada (`targets`, como `aarch64-apple-darwin`, `x86_64-unknown-linux-gnu` o `wasm32-unknown-unknown`). Todo paquete en Rust se denomina *crate* (binario o biblioteca) y se gobierna mediante el manifiesto declarativo `Cargo.toml`. Mientras que `Cargo.toml` expresa rangos semánticos compatibles (SemVer, por ejemplo `serde = "1.0"`), el fichero `Cargo.lock` registra las versiones exactas y los resúmenes criptográficos SHA-256 de cada dependencia, garantizando compilaciones bit a bit reproducibles. Los comandos esenciales del ciclo de ingeniería son `cargo check` (verificación rápida de tipos y préstamos en MIR sin invocar el generador de código LLVM), `cargo test` (ejecución concurrente de pruebas unitarias, de integración y de documentación), `cargo clippy - -D warnings` (análisis estático con más de 700 reglas idiomáticas y de rendimiento) y `cargo fmt -check`.

2.2 Bloque C: Variables, tipos primitivos, expresiones de control de flujo y funciones

En Rust, las vinculaciones introducidas con `let x: i32 = 42;` son inmutables por defecto; la mutabilidad requiere la marca explícita `let mut contador: u64 = 0;`. El lenguaje proporciona enteros con signo (`i8`, `i16`, `i32`, `i64`, `i128`, `isize`), enteros sin signo (`u8`, `u16`, `u32`, `u64`, `u128`, `usize`), flotantes IEEE 754 (`f32`, `f64`), el tipo booleano `bool` (1 byte) y el tipo `char` de 4 bytes que representa exactamente un valor escalar Unicode (U+0000 a U+D7FF y U+E000 a U+10FFFF). En caso de desbordamiento entero, el perfil `dev` genera un *panic* determinista, mientras que el lenguaje ofrece métodos explícitos para cada semántica aritmética: `checked_add` (devuelve `Option<T>`), `wrapping_add` (aritmética modular mód 2^w), `saturating_add` (satura en los extremos) y `overflowing_add`:

```
pub fn classify_http_status(code: u16) -> &'static str {
    match code {
        100..=199 => "informational",
        200..=299 => "success",
        300..=399 => "redirection",
        400..=499 => "client_error",
        500..=599 => "server_error",
        _ => "unknown",
    }
}
```

Observa que `match` exige exhaustividad verificada por el compilador: si se omitiera el brazo comodín `_`, `rustc` rechazaría el programa indicando exactamente los rangos de `u16` no cubiertos (`0..=99` y `600..=65535`).

Problema resuelto

Enunciado Una cadena en Rust contiene 10 caracteres ASCII (1 B cada uno en UTF-8) y 5 caracteres acentuados españoles (`á`, `é`, `í`, `ó`, `ú`, que ocupan 2 B cada uno en UTF-8). Calcula la diferencia en bytes entre representarla como `&str` en UTF-8 frente a un array `[char; 15]` de valores escalares Unicode (4 B por `char`).

Datos

- $N_{ascii} = 10$ — Caracteres ASCII de 1 byte en UTF-8
- $N_{lat} = 5$ — Caracteres latinos acentuados de 2 bytes en UTF-8
- $S_{char} = 4\,\mathrm{B}$ — Tamaño fijo del tipo char en Rust

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Comparación de almacenamiento UTF-8 (&str) frente a array de escalares Unicode [char; N]

$$\Delta B = 15 \cdot 4 - (10 \cdot 1 + 5 \cdot 2)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 20 bytes de la codificación UTF-8 a los 60 bytes del array [char; 15]

$$\Delta B = 15 \cdot 4 - (10 \cdot 1 + 5 \cdot 2)$$

$$((15 \cdot 4) - ((10 \cdot 1) + (5 \cdot 2)))$$

Resultado

$$Resultado = 40. La representación UTF-8 nativa de Rusta ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String$$

Interpretación La representación UTF-8 nativa de Rust ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String y &str utilizan UTF-8 en lugar de arrays de 32 bits.

Comprobación de los pasos

- Paso 1: 40 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0011] (Básico, Numérico) Calcula el tamaño en bytes de un array `[u64; 32]` alojado en la pila (32×8).
2 [PRB-0012] (Básico, Desarrollo por pasos) Una cadena UTF-8 contiene 8 caracteres ASCII (1 B), 4 letras griegas (2 B) y 2 emojis (4 B). ¿Cuál es el valor de `s.len()` en bytes?
3 [PRB-0013] (Intermedio, Desarrollo por pasos) Calcula el resultado de `250u8.wrapping_add(15)` en aritmética modular de 8 bits ($250 + 15 - 256$).
4 [PRB-0014] (Intermedio, Diseño) Si `Vec::with_capacity(128)` reserva espacio para 128 elementos `i32` (4 B cada uno) e insertamos 50 elementos, ¿cuántos bytes de capacidad reservada permanecen aún libres en el montículo?

Parte II

Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad

Capítulo 3

D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

*Este capítulo desarrolla el corazón teórico y práctico de Rust: su sistema de tipos afín basado en propiedad única (*ownership*), préstamo (*borrowing*) y regiones de tiempo de vida (*lifetimes*). Comprender este modelo no solo permite escribir Rust sin fricción, sino que transforma permanentemente la manera en que un ingeniero razona sobre la vida útil de los recursos y la concurrencia en cualquier lenguaje.*

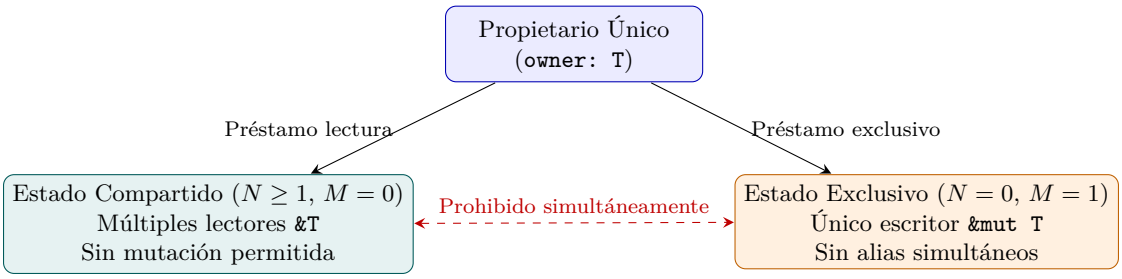


Figura 2. Invariante fundamental del comprobador de préstamos: en cualquier punto del flujo de control MIR, un recurso admite múltiples préstamos inmutables (&T) o exactamente un préstamo exclusivo mutable (&mut T).

Consejo

Recuerda que las anotaciones `'a` nunca prolongan la vida de un valor en memoria: únicamente declaran al compilador cómo se relaciona la vida útil de la referencia de salida con la de las referencias de entrada.

Definición

Parámetro de tiempo de vida `'a`: variable de región en el sistema de tipos de Rust que restringe estáticamente el ámbito durante el cual una referencia prestada `&'a T` o `&'a mut T` es válida, garantizando que el referente sobrevive a toda referencia.

Autoficha

¿Por qué la firma `fn dangling() -> &str { let s = String::from("hola"); &s }` es rechazada en compilación independientemente del tiempo de vida que se intente anotar?

3.1 Las tres leyes de Ownership, semántica Move frente a Copy y el rasgo Drop

El sistema de propiedad de Rust se rige por tres axiomas verificados en compilación: (1) cada valor en Rust tiene una variable denominada su *propietario* (*owner*); (2) solo puede existir un único propietario a la vez; y (3) cuando el propietario sale de su ámbito léxico (*scope*), el valor se destruye invocando determinísticamente `Drop::drop` (patrón RAII, *Resource Acquisition Is Initialization*). Cuando asignamos o pasamos por valor un tipo que gestiona memoria en el montículo como `String` o `Vec<T>`, Rust ejecuta una transferencia de propiedad (*move*): copia los 24 bytes de la cabecera en la pila e invalida estáticamente la variable de origen en el grafo de flujo MIR, impidiendo cualquier doble liberación (*double free*). Por el contrario, los tipos escalares simples que residen íntegramente en la pila implementan el

rasgo marcador `Copy`, por lo que la asignación duplica sus bits sin invalidar el origen. Un tipo solo puede implementar `Copy` si ninguno de sus componentes implementa `Drop`.

3.2 Reglas de Borrowing (&T vs &mut T), Non-Lexical Lifetimes (NLL) y parámetros de vida 'a

Para operar sobre un valor sin transferir su propiedad, Rust introduce las referencias (*borrowing*). Como muestra la Figura \ref{fig:fig-rust-aliasing-xor-mutability}, en cualquier punto de ejecución del grafo de control MIR para un recurso dado, el número de referencias inmutables activas N_{shared} y el número de referencias mutables activas M_{mut} satisfacen la invariante de exclusión:

$$(N_{\text{shared}} \geq 0 \land M_{\text{mut}} = 0) \lor (N_{\text{shared}} = 0 \land M_{\text{mut}} = 1)$$

Esta ley de *Alias XOR Mutabilidad* elimina de raíz tanto las carreras de datos entre hilos como la invalidación de iteradores en un único hilo (por ejemplo, intentar hacer `vec.push(x)` mientras se mantiene una referencia `&vec[0]` que quedaría colgante si el vector realoja su búfer en el montículo). Cuando una función devuelve una referencia derivada de sus parámetros de entrada, el parámetro de tiempo de vida `'a` expresa la relación de inclusión de regiones $'a \subseteq \text{vida}(x) \cap \text{vida}(y)$, sin alterar jamás la duración real en tiempo de ejecución:

```
pub fn longest_token<'a>(left: &'a str, right: &'a str) -> &'a str {
    if left.len() >= right.len() { left } else { right }
}
```

El compilador aplica tres reglas deterministas de *elisión de tiempos de vida*: (1) cada parámetro por referencia recibe su propio parámetro `'a`, `'b`; (2) si hay exactamente un tiempo de vida de entrada, se asigna a todas las referencias de salida; y (3) si uno de los parámetros es `&self` o `&mut self`, su tiempo de vida se asigna a todas las referencias de salida.

Problema resuelto

Enunciado Un vector `Vec<u64>` contiene $N = 10000$ enteros de 8 bytes en el montículo en una máquina de 64 bits ($W = 8$ B). Calcula cuántos bytes menos se copian al pasar el vector por transferencia de propiedad (*move*, que solo copia las 3 palabras de la cabecera en pila) frente a invocar `.clone()` (que copia la cabecera más los 10000×8 bytes del montículo).

Datos

- $N = 10000$ — Número de elementos u64 en el vector
- $S_T = 8\,\text{B}$ — Tamaño de cada elemento u64
- $W = 8\,\text{B}$ — Tamaño de palabra en arquitectura de 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Diferencia de bytes copiados entre Move (O(1)) y Clone (O(N))

$$\Delta B = (3 \cdot 8 + 10000 \cdot 8) - 3 \cdot 8$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el coste de move (24 B) al coste total de clone (80024 B)

$$\Delta B = (3 \cdot 8 + 10000 \cdot 8) - 3 \cdot 8$$

$$(((3 \cdot 8) + (10000 \cdot 8)) - (3 \cdot 8))$$

Resultado

$Resultado = 80000. La transferencia de propiedad (move) ahorra la copia de 80\,000 bytes en el montículo y se ejecuta en tiempo constante O(1) copiando únicamente 24 bytes.$

Comprobación de los pasos

- Paso 1: 80000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0018] (Básico, Numérico) Calcula cuántos bytes en la pila se copian al mover un `String` en una arquitectura de 64 bits (3×8).
2 [PRB-0019] (Básico, Numérico) Si dividimos una rebanada de 1024 elementos mediante `split_at_mut(300)`, ¿cuántos elementos contiene la segunda sub-rebanada mutable?
3 [PRB-0020] (Intermedio, Desarrollo por pasos) Una estructura contiene 4 campos `u64` que implementan `Copy`. ¿Cuántos bytes se copian en cada asignación por valor (4×8)?
4 [PRB-0021] (Intermedio, Diseño) Un analizador léxico de copia cero (*zero-copy*) genera 500 tokens `&'a str` (16 B cada uno) sobre un texto fuente de 32 000 bytes sin clonar subcadenas. ¿Cuántos bytes ocupan en total los 500 descriptores de token?

Capítulo 4

D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

*Este capítulo completa el Nivel Intermedio (Bloque D) profundizando en el sistema de tipos algebraicos, el polimorfismo basado en rasgos (*traits*) y la arquitectura de manejo de errores de Rust. A diferencia de los lenguajes con referencias nulas implícitas o excepciones no comprobadas, Rust codifica la ausencia de valor (`Option<T>`) y la posibilidad de fallo (`Result<T, E>`) directamente en las firmas de tipos.*

Consejo

En bibliotecas reutilizables, define tipos de error estructurados como `enum` que implementen `std::error::Error` y `Display` (o utiliza `thiserror`), reservando errores opacos tipo `Box<dyn Error>` o `anyhow` para la capa superior binaria.

Definición

Optimización de nicho (Niche Optimization): transformación de representación mediante la cual el compilador de Rust codifica variantes de un tipo suma (`enum`) utilizando patrones de bits inválidos (nichos) del tipo contenido, eliminando la necesidad de un byte discriminante adicional.

Autoficha

¿Cuándo debe un rasgo utilizar un tipo asociado (`trait Iterator { type Item; }`) en lugar de un parámetro de tipo genérico (`trait From<T>`)?

4.1 Estructuras, enumeraciones algebraicas, optimización de nicho y coincidencia de patrones

Un `struct` en Rust define un tipo producto cuya cardinalidad es el producto de los dominios de sus campos, mientras que un `enum` define un tipo suma etiquetado (*tagged union*) donde cada variante puede transportar datos de tipos distintos: por ejemplo, `enum IpPacket { V4([u8; 4], u16), V6([u16; 8], u16), Dropped }`. El tamaño en memoria de un `enum` es el tamaño de su variante más grande más el discriminante (etiqueta), redondeado al múltiplo del alineamiento máximo. Sin embargo, el compilador aplica la *optimización de nicho* (*null pointer / niche optimization*): como una referencia `&T`, un `Box<T>` o un `NonZeroU64` jamás pueden tener el patrón de bits cero, `Option<&T>` utiliza el valor cero para representar `None`, por lo que sizeof(Option$\langle \&T \rangle$) = sizeof($\&T$) = 8 B, con sobrecoste espacial exactamente nulo.

4.2 Rasgos (Traits), genéricos, tipos asociados y manejo de errores con Option<T> y Result<T, E>

Los rasgos (*traits*) definen interfaces de comportamiento compartido que admiten implementaciones por defecto, parámetros genéricos y tipos asociados (`type Item;`). Para garantizar la coherencia global de tipos al enlazar múltiples crates, Rust impone la *regla del huérfano* (*orphan rule*): solo se puede implementar un rasgo `Trait` para un tipo `Type` en el crate actual si al menos uno de los dos (`Trait` o `Type`) está definido localmente en dicho crate. En el tratamiento de errores, `Result<T, E>` se combina con el operador de propagación `?`, que desazucara una expresión `expr?` en una evaluación exhaustiva que convierte automáticamente el error mediante `From::from(err)`:

```
#[derive(Debug, PartialEq)]
pub enum ConfigError { EmptyInput, InvalidPort(String) }

pub fn parse_port_str(raw: &str) -> Result<u16, ConfigError> {
    let trimmed = raw.trim();
    if trimmed.is_empty() {
        return Err(ConfigError::EmptyInput);
    }
    let port: u16 = trimmed.parse().map_err(|_| ConfigError::InvalidPort(trimmed.to_string()))
    ?;
    if port == 0 { Err(ConfigError::InvalidPort(trimmed.to_string())) } else { Ok(port) }
}
```

Esta disciplina distingue con nitidez entre errores recuperables (modelados con `Result<T, E>`) y violaciones de invariantes de programación irrecuperables (modeladas con `panic!`).

Problema resuelto

Enunciado Una tabla hash abierta almacena $N = 65536$ ranuras de tipo `Option<std::num::NonZeroU64>` en una arquitectura de 64 bits ($W = 8$ B). Gracias a la optimización de nicho, cada `Option<NonZeroU64>` ocupa exactamente 8 bytes en lugar de 16 bytes (8 bytes de dato más 8 bytes de discriminante alineado). Calcula cuántos kilobytes (dividiendo entre 1024) se ahorran en la tabla completa gracias a la optimización de nicho.

Datos

- $N = 65536$ — Número de ranuras en la tabla
- $S_{sin} = 16\,\text{B}$ — Tamaño alineado sin optimización de nicho
- $S_{con} = 8\,\text{B}$ — Tamaño con optimización de nicho

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria por optimización de nicho en Option<NonZeroU64>

$$\Delta M_{\text{KiB}} = \frac{65536 \cdot (16 - 8)}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 65536 ranuras por los 8 B ahorrados y dividimos entre 1024 B/KiB

$$\Delta M_{\text{KiB}} = \frac{65536 \cdot (16 - 8)}{1024}$$

$$\frac{(65536 \cdot (16 - 8))}{1024}$$

Resultado

$$Resultado = 512. La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entr$$

Interpretación La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entradas por línea de caché de 64 bytes.

Comprobación de los pasos

- Paso 1: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0025] (Básico, Numérico) En una arquitectura de 64 bits, ¿cuántos bytes ocupa `Option<Box<u64>>` gracias a la optimización de nicho?
2 [PRB-0026] (Básico, Numérico) ¿Cuántos bytes ocupa en total un array `[Option<NonZeroU32>; 1000]` sabiendo que cada elemento ocupa 4 bytes?
3 [PRB-0027] (Intermedio, Desarrollo por pasos) Un `enum` sin nicho tiene una variante de 24 bytes con alineamiento de 8 bytes y una etiqueta de 1 byte. Tras añadir 7 bytes de relleno (*padding*) para mantener el alineamiento a múltiplo de 8, ¿cuál es su tamaño total en bytes?
4 [PRB-0028] (Intermedio, Diseño) Una línea de caché de CPU mide 64 bytes. ¿Cuántos elementos `Option<NonZeroU64>` (8 B cada uno) caben en una única línea de caché?

Parte III

Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe

Capítulo 5

E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

Este capítulo abre el Nivel Avanzado (Bloque E). Estudiaremos cómo Rust implementa la programación funcional mediante estructuras anónimas de clausura e iteradores fusionados en compilación, y cómo permite relajar las restricciones estáticas de propiedad única cuando un algoritmo requiere grafos compartidos o cachés internas mediante punteros inteligentes y contenedores de mutabilidad interior.

Consejo

Para evitar fugas de memoria en estructuras doblemente enlazadas o árboles con puntero al nodo padre, utiliza `Rc<Node>` de padre a hijo y `Weak<Node>` de hijo a padre, ya que `Weak<T>` no incrementa el `strong_count`.

Definición

Mutabilidad interior (Interior Mutability): patrón de diseño seguro en Rust mediante el cual una estructura expone una interfaz inmutable (`&self`) pero permite mutar su estado interno encapsulado en `UnsafeCell<T>` garantizando la invariante de exclusión mediante transferencia de valores (`Cell<T>`), contadores dinámicos (`RefCell<T>`) o sincronización atómica (`Mutex<T>`, `RwLock<T>`).

Autoficha

¿Por qué `Rc<T>` no implementa el rasgo `Send` y qué ocurriría en hardware multinúcleo si dos hilos ejecutaran `Rc::clone` simultáneamente sobre el mismo contador no atómico?

5.1 Clausuras (Fn, FnMut, FnOnce) e iteradores perezosos de coste cero

Cada clausura `|x| x + umbral` en Rust genera un tipo `struct` anónimo único que captura las variables libres de su entorno por el modo menos restrictivo posible: por referencia compartida `&T` (implementa `Fn`, `FnMut` y `FnOnce`), por referencia mutable `&mut T` (implementa `FnMut` y `FnOnce`) o por valor consumiendo la variable (implementa únicamente `FnOnce`, o cuando se fuerza la captura con la palabra clave `move`). Los iteradores en Rust implementan el rasgo `Iterator { type Item; fn next(&mut self) -> Option<Self::Item>; }`. Adaptadores como `.map(...)`, `.filter(...)`, `.take(...)` y `.enumerate()` son perezosos (*lazy*): no ejecutan ningún trabajo ni alojan vectores intermedios hasta que un consumidor terminal como `.fold(...)`, `.sum()` o `.collect::<Vec<_>>()` solicita los elementos. Gracias a la monomorfización, LLVM fusiona toda la cadena de adaptadores en un único bucle registrador sin comprobaciones de límites redundantes.

5.2 Punteros inteligentes (Box, Rc, Weak) y mutabilidad interior (Cell, RefCell, Cow)

Un puntero inteligente (*smart pointer*) es una estructura que implementa los rasgos `Deref` y `Drop`. `Box<T>` proporciona propiedad única en el montículo (indispensable para tipos recursivos de tamaño finito como árboles sintácticos `enum Expr { Add(Box<Expr>, Box<Expr>), Lit(i64) }`). `Rc<T>` (*Reference Counted*) habilita propiedad compartida en un único hilo manteniendo un contador de referencias fuertes (`strong_count`) y débiles (`weak_count`); como `Rc<T>` no utiliza instrucciones atómicas, el compilador le

niega los rasgos `Send` y `Sync`, impidiendo estáticamente que cruce a otro hilo. Cuando se necesita mutar un dato a través de una referencia compartida `&self`, Rust ofrece *mutabilidad interior*: `Cell<T>` opera por reemplazo de valores sin referencias internas (coste cero en espacio), mientras que `RefCell<T>` traslada las reglas del comprobador de préstamos al tiempo de ejecución manteniendo un contador de préstamos (`borrow()` e `borrow_mut()`):

```
use std::cell::RefCell;
use std::rc::Rc;

pub fn shared_accumulator_demo() -> i64 {
    let acc = Rc::new(RefCell::new(10_i64));
    let handle_a = Rc::clone(&acc);
    let handle_b = Rc::clone(&acc);
    *handle_a.borrow_mut() += 15;
    *handle_b.borrow_mut() *= 2;
    let final_val = *acc.borrow();
    final_val
}
```

Si un programa intenta adquirir un `borrow_mut()` mientras otro préstamo de la misma `RefCell<T>` sigue vivo, `RefCell` dispara un *panic* inmediato en lugar de permitir corrupción de memoria.

Problema resuelto

Enunciado En una arquitectura de 64 bits ($W = 8$ B), se aloja un bloque `Rc< [u8; 1024] >` ($S_T = 1024$ B) y se crean $k = 5$ manejadores `Rc::clone` en distintas estructuras en la pila. Calcula la memoria total consumida (los 5 punteros en la pila más el único bloque `RcBox` en el montículo con sus 2 contadores `usize` y los 1024 bytes de datos).

Datos

- $W = 8\,\mathrm{B}$ — Tamaño de usize y puntero en 64 bits
- $k = 5$ — Número de manejadores Rc<T> en la pila
- $S_T = 1024\,\mathrm{B}$ — Tamaño del valor compartido en el montículo

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Consumo de memoria de Rc<T> con k referencias compartidas

$$M = 5 \cdot 8 + (2 \cdot 8 + 1024)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los 5 punteros de 8 B en pila más el RcBox (16 B de contadores + 1024 B de datos)

$$M = 5 \cdot 8 + (2 \cdot 8 + 1024)$$

$$((5 \cdot 8) + ((2 \cdot 8) + 1024))$$

Resultado

$Resultado = 1080. El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente$

Interpretación El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente a clonar 5 veces el búfer.

Comprobación de los pasos

- Paso 1: 1080 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0032] (Básico, Numérico) Calcula la suma de los cuadrados de los números pares del 2 al 10 ($2^2 + 4^2 + 6^2 + 8^2 + 10^2$) evaluada por `(1..=10).filter(|x| x% 2 == 0).map(|x| x * x).sum()`.

2 [PRB-0033] (Básico, Numérico) ¿Cuántos bytes de metadatos añaden los dos contadores (`strong` y `weak`) de `RcBox<T>` en una arquitectura de 64 bits (2×8)?

3 [PRB-0034] (Intermedio, Desarrollo por pasos) Si se crean 1 `Rc` original, 4 clones fuertes (`Rc::clone`) y 3 referencias débiles (`Rc::downgrade`), y luego se destruyen 2 clones fuertes, ¿cuál es el valor actual de `Rc::strong_count`?

4 [PRB-0035] (Intermedio, Diseño) Un flujo de datos procesa 5000 enteros mediante `iter().filter(...).map(...).fold(...)` sin llamar a `collect()`. ¿Cuántos vectores dinámicos intermedios se alojan en el montículo durante el recorrido?

Capítulo 6

E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

Este capítulo aborda la concurrencia paralela y la asincronía de alto rendimiento en Rust. Veremos por qué el mismo sistema de tipos que previene punteros colgantes en un hilo es capaz de impedir toda carrera de datos entre múltiples hilos mediante los rasgos marcadores `Send` *y* `Sync`*, y cómo Rust compila funciones* `async fn` *en máquinas de estados compactas sin requerir un alojamiento en el montículo por cada llamada asíncrona.*

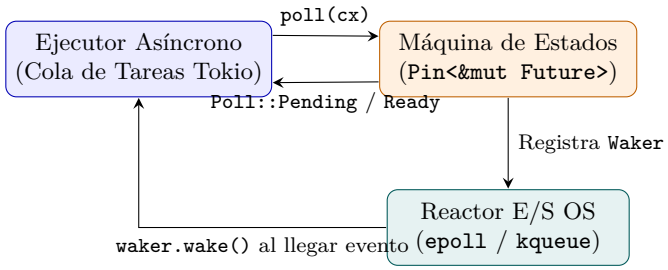


Figura 3. Modelo de ejecución de Future en Rust: el ejecutor (Tokio) invoca poll(Pin<&mut Self>, &mut Context); si el recurso E/S no está listo devuelve Poll::Pending y registra el Waker en el reactor del sistema operativo (epoll/kqueue).

Consejo

En código asíncrono sobre Tokio, nunca mantengas bloqueado un `std::sync::MutexGuard` a través de un punto `.await` ni ejecutes cómputo intensivo de CPU sin ceder el control; usa bloques cortos sin `.await` o delega el cómputo pesado a `tokio::task::spawn_blocking`.

Definición

Fijación de dirección (`Pin<P>`): contrato del sistema de tipos de Rust que garantiza que el valor apuntado por el puntero `P` no será movido jamás a otra dirección de memoria mientras no implemente `Unpin`, hasta que se ejecute su destructor `Drop::drop`.

Autoficha

¿Por qué en Rust un `Mutex<T>` es un contenedor genérico que envuelve al dato `T` en lugar de ser un objeto cerrojo separado del dato como en C++ (`std::mutex`) o Java?

6.1 Concurrencia basada en hilos: Send, Sync, thread::scope, Arc, Mutex y canales

Dos rasgos marcadores automáticos gobiernan toda la concurrencia en Rust: un tipo `T` es `Send` si su propiedad puede transferirse de manera segura a otro hilo, y es `Sync` si es seguro compartirlo entre varios hilos simultáneamente por referencia inmutable (es decir, `T: Sync` si y solo si `&T: Send`). Mientras que `std::thread::spawn` exige una clausura `'static + Send` porque el hilo hijo podría sobrevivir a la función que lo creó, `std::thread::scope` permite lanzar hilos que toman prestadas referencias locales no `'static` del marco de pila padre, garantizando estáticamente que todos los hilos del ámbito terminan antes de que `scope` retorne:

```
use std::thread;

pub fn parallel_dot_product(a: &[i64], b: &[i64]) -> i64 {
    let mid = a.len() / 2;
    let (a_left, a_right) = a.split_at(mid);
    let (b_left, b_right) = b.split_at(mid);
    thread::scope(|s| {
        let left_handle = s.spawn(|| a_left.iter().zip(b_left).map(|(x, y)| x * y).sum::<i64
    >());
        let right_sum: i64 = a_right.iter().zip(b_right).map(|(x, y)| x * y).sum();
        left_handle.join().unwrap() + right_sum
    })
}
```

Cuando varios hilos deben compartir y mutar estado sin un ámbito estático común, se combina el contador atómico de referencias `Arc<T>` con un cerrojo que envuelve directamente al dato protegido (`Mutex<T>` o `RwLock<T>`): en Rust es sintácticamente imposible acceder al dato sin adquirir primero el `MutexGuard`.

6.2 Programación asíncrona: el rasgo Future, máquinas de estados, Pin/Unpin y el ejecutor Tokio

Para cargas masivas de entrada/salida de red con cientos de miles de conexiones simultáneas, crear un hilo del sistema operativo por conexión resulta prohibitivo. En lugar de ello, una función `async fn` en Rust devuelve un valor que implementa el rasgo `Future { type Output; fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>; }`. Como ilustra la Figura \ref{fig:fig-rust-async-poll-state-machine}, los futuros en Rust son perezosos y dirigidos por sondeo (*poll-driven*): el compilador transforma el cuerpo de una `async fn` en un `enum` donde cada punto `.await` es un estado que guarda únicamente las variables vivas a través de esa suspensión. Dado que una variable local puede ser prestada a través de un punto `.await`, la máquina de estados generada es *autorreferencial* (un campo del `enum` contiene un puntero a otro campo del mismo `enum`). Si esa estructura se moviera de dirección en memoria después del primer `poll`, el puntero interno quedaría colgante; por ello `Future::poll` recibe `self: Pin<&mut Self>`, un envoltorio de tipos que prohíbe obtener `&mut T` para mover el valor fuera de su dirección fijada salvo que el tipo implemente `Unpin`.

Problema resuelto

Enunciado Un servidor de mensajería mantiene $C = 50000$ conexiones TCP simultáneas. Si cada máquina de estados `Future` en Tokio ocupa 256 B (0,25 KiB) mientras que una pila mínima de hilo cuesta 64 KiB, calcula la memoria total en MiB (dividiendo los KiB entre 1024) que consumen las 50 000 tareas asíncronas.

Datos

- $C = 50000$ — Conexiones concurrentes activas
- $S_{future} = 256\,\mathrm{B}$ — Tamaño de cada máquina de estados Future
- $B_{MiB} = 1048576\,\mathrm{B\,MiB^{-1}}$ — Bytes por mebibyte (1024 * 1024)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Huella de memoria en MiB de C máquinas de estados Future

$$M_{\mathrm{MiB}} = \frac{50000 \cdot 256}{1024 \cdot 1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 50000 tareas por 256 B y dividimos entre 1024*1024

$$M_{\text{MiB}} = \frac{50000 \cdot 256}{1024 \cdot 1024}$$

$$\frac{(50000 \cdot 256)}{(1024 \cdot 1024)}$$

Resultado

$Resultado = 12{,}20703125. Las 50000 conexiones asíncronas consumen tan solo 12{,}20703125 MiB de RAM, frente a los 3125 M$

Interpretación Las 50 000 conexiones asíncronas consumen tan solo 12.20703125 MiB de RAM, frente a los 3125 MiB (> 3 GiB) que exigirían 50 000 pilas de 64 KiB.
Comprobación de los pasos

- Paso 1: 12,207 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) Cuatro hilos incrementan un `AtomicU64` exactamente 25 000 veces cada uno con `fetch_add(1, Ordering::Relaxed)`. ¿Cuál es el valor final exacto del contador?
2 [PRB-0040] (Básico, Numérico) Un vector de 10 000 elementos se divide equitativamente entre 8 hilos mediante `chunks(1250)` dentro de `std::thread::scope`. ¿Cuántos elementos procesa cada hilo?
3 [PRB-0041] (Intermedio, Desarrollo por pasos) Una `async fn` tiene 3 variables locales `u64` (8 B cada una) vivas simultáneamente en su punto `.await` más grande, más 8 bytes de discriminante de estado. ¿Cuántos bytes ocupa esa máquina de estados `Future`?
4 [PRB-0042] (Intermedio, Diseño) Un canal acotado `mpsc::sync_channel(64)` almacena mensajes de 128 bytes cada uno. ¿Cuántos bytes como máximo ocupan los mensajes en tránsito cuando el búfer está lleno?

Capítulo 7

F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

Este capítulo desarrolla el Nivel Experto (Bloque F). Cuando un ingeniero de sistemas necesita implementar nuevas primitivas de sincronización, estructuras de datos intrusivas, controladores de registros de hardware mapeados en memoria (MMIO) o enlazar con bibliotecas en C del sistema operativo, recurre al subconjunto `unsafe` de Rust bajo contratos formales auditables con `Miri`.

Consejo

Todo bloque `unsafe { ... }` debe ir precedido por un comentario `// SAFETY:` que demuestre formalmente por qué se cumplen las precondiciones en ese punto, y toda función `pub unsafe fn` debe incluir una sección `# Safety` en su documentación.

Definición

Invariante de validez frente a invariante de seguridad en Rust: la invariante de validez exige que un tipo nunca adopte un patrón de bits ilegal (por ejemplo, un `bool` distinto de 0 o 1, o una referencia `&T` nula o desalineada), mientras que la invariante de seguridad exige que ninguna secuencia de llamadas desde código seguro pueda inducir comportamiento indefinido.

Autoficha

¿Por qué el compilador de Rust reordena por defecto los campos de un `struct` (`repr(Rust)`) y por qué es obligatorio añadir `#[repr(C)]` antes de pasar un puntero a esa estructura a una función escrita en C?

7.1 Rust inseguro (unsafe), contratos de seguridad, Miri e interoperabilidad FFI con C

Un bloque `unsafe { ... }` en Rust **no** desactiva el comprobador de préstamos ni las reglas de propiedad sobre referencias ordinarias (`&T` y `&mut T`); únicamente habilita cinco operaciones adicionales cuyos invariantes de validez no puede demostrar estáticamente el compilador: (1) desreferenciar punteros crudos (`*const T` y `*mut T`); (2) llamar a una función o método marcado como `unsafe fn`; (3) acceder o modificar una variable estática mutable; (4) implementar un rasgo marcado como `unsafe trait` (como `Send` o `Sync`); y (5) acceder a campos de una `union`. La arquitectura idiomática de Rust encapsula todo bloque `unsafe` dentro de un módulo con fronteras pequeñas y expone al exterior una API 100 % segura:

```
#[repr(C)]
#[derive(Copy, Clone, Debug, PartialEq)]
pub struct CTelemetryHeader {
    pub magic: u32,
    pub version: u16,
    pub payload_len: u16,
}

/// Divide una rebanada mutable en dos partes disjuntas usando punteros crudos.
pub fn safe_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = slice.len();
    assert!(mid <= len, "mid fuera de rango");
    let ptr = slice.as_mut_ptr();
```

```
    // SAFETY: 0 <= mid <= len garantiza que [ptr, ptr+mid) y [ptr+mid, ptr+len)
    // están dentro de la misma asignación y son estrictamente disjuntos.
    unsafe {
        (std::slice::from_raw_parts_mut(ptr, mid),
         std::slice::from_raw_parts_mut(ptr.add(mid), len - mid))
    }
}
```

Para interoperar con C mediante FFI (*Foreign Function Interface*), se utiliza `#[repr(C)]` (ya que el diseño por defecto `repr(Rust)` reordena libremente los campos de un `struct` para minimizar el relleno de alineamiento) y la convención de llamada `extern "C"`. El intérprete abstracto `cargo miri test` ejecuta el código MIR verificando el modelo de alias *Stacked Borrows* / *Tree Borrows*, lecturas de memoria no inicializada y alineamientos inválidos.

7.2 Metaprogramación con macros, entornos empotrados (#![no_std]), WebAssembly y optimización SIMD

En metaprogramación, Rust ofrece dos familias de macros que operan sobre el árbol sintáctico antes de la comprobación de tipos: las macros declarativas (`macro_rules!`), que funcionan por coincidencia de patrones higiénica sobre fragmentos sintácticos (`expr`, `ty`, `ident`, `tt`), y las macros procedimentales (`#[derive(...)]`, macros de atributo y macros tipo función), que son crates compilados que transforman un flujo `TokenStream` en otro `TokenStream`. En el ámbito de sistemas empotrados y kernels de sistemas operativos, el atributo `#![no_std]` prescinde de la biblioteca estándar `std` (que depende de un sistema operativo subyacente) y opera únicamente sobre `core` (independiente de plataforma y sin montículo) y opcionalmente `alloc`. Del mismo modo, Rust compila de forma nativa a `wasm32-unknown-unknown` y `wasm32-wasip1`, permitiendo ejecutar motores criptográficos, analizadores y simuladores físicos en el navegador o en pasarelas *serverless* a velocidad cercana al código nativo.

Problema resuelto

Enunciado Una estructura declarada con `#[repr(C)]` contiene en este orden exacto: `a: u8` (1 B), `b: u64` (8 B, requiere alineamiento a múltiplo de 8), `c: u8` (1 B) y `d: u32` (4 B, requiere alineamiento a múltiplo de 4), con alineamiento total de la estructura a múltiplo de 8 bytes. Calcula la diferencia en bytes para un array de $N = 1000$ estructuras entre `#[repr(C)]` (24 B cada una) y el empaquetado óptimo de `repr(Rust)` que ordena `b: u64`, `d: u32`, `a: u8`, `c: u8` ($8+4+1+1+2$ padding $= 16$ B).
Datos

- $N = 1000$ — Número de estructuras en el array
- $S_C = 24\,\text{B}$ — Tamaño bajo #[repr(C)] (1+7 + 8 + 1+3 + 4 = 24 B)
- $S_{Rust} = 16\,\text{B}$ — Tamaño reordenado por repr(Rust) (8 + 4 + 1 + 1 + 2 = 16 B)

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de relleno (padding) de alineamiento entre repr(C) y repr(Rust)

$$\Delta B = 1000 \cdot (24 - 16)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 1000 instancias por los 8 B ahorrados por estructura

$$\Delta B = 1000 \cdot (24 - 16)$$

$$(1000 \cdot (24 - 16))$$

Resultado

$Resultado = 8000. La reordenación automática de campos de repr(Rust) ahorra 8000 bytes (un 33{,}3\,\% de reducción de huella en caché) sin requerir acceso desalineado.$

Interpretación La reordenación automática de campos de repr(Rust) ahorra 8000 bytes (un 33.3 % de reducción de huella en caché) sin requerir acceso desalineado.

Comprobación de los pasos

- Paso 1: 8000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0046**] **(Básico, Numérico)** Calcula el tamaño en bytes de `BadLayoutC { a: u8, b: u64, c: u8 }` bajo `#[repr(C)]` en 64 bits ($1 + 7 + 8 + 1 + 7$).
2 [**PRB-0047**] **(Básico, Numérico)** Calcula el tamaño en bytes de `GoodLayoutRust { a: u8, b: u64, c: u8 }` reordenado por el compilador de Rust ($8 + 1 + 1 + 6$).
3 [**PRB-0048**] **(Intermedio, Desarrollo por pasos)** Un registro SIMD de 256 bits (`AVX2`) procesa carriles `f32` de 32 bits en paralelo. ¿Cuántos números `f32` opera en una sola instrucción?
4 [**PRB-0049**] **(Intermedio, Diseño)** En un microcontrolador ARM Cortex-M `#![no_std]` con 64 KiB de SRAM (65536 B), el firmware reserva 8192 bytes para la pila y 4096 bytes para variables estáticas. ¿Cuántos bytes quedan disponibles para buffers estáticos de DMA?

Parte IV

Bloques G–H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos

Capítulo 8

G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

*Este capítulo culmina la monografía de Rust cubriendo los Bloques G y H de la plantilla: el ecosistema real de producción, los patrones de diseño de APIs conformes a las *Rust API Guidelines*, el catálogo de antipatrones frecuentes y la construcción paso a paso de cuatro proyectos completos, culminando en el motor de almacenamiento Key-Value transaccional con registro de escritura anticipada (*Write-Ahead Log*) y suma de verificación CRC32 ubicado en* `projects/rust-kv-engine`.

Consejo

Nunca expongas `&String`, `&Vec<T>` ni `&Box<T>` en los parámetros de entrada de funciones públicas: utiliza `&str`, `&[T]` y `&T`, ya que las coerciones automáticas `Deref` permiten pasar tanto tipos propietarios como vistas prestadas sin coste.

Definición

Write-Ahead Log (WAL) con verificación CRC32: estructura de almacenamiento secuencial sólo-añadir (append-only) en la que toda operación de mutación se persiste en disco precedida de un código de redundancia cíclica antes de confirmarse en la estructura de datos en memoria, garantizando atomicidad y durabilidad (propiedades A y D de ACID).

Autoficha

¿Por qué las *feature flags* de un manifiesto `Cargo.toml` deben ser siempre aditivas y qué problema ocurre cuando dos dependencias de un mismo workspace activan combinaciones distintas de una feature no aditiva?

8.1 Bloque G: Ecosistema de crates, Cargo Workspaces, idioms de diseño de APIs y antipatrones

El ecosistema industrial de Rust se articula en torno a crates fundacionales de altísima cohesión: `serde` (serialización y deserialización de coste cero sin reflexión en tiempo de ejecución mediante generación de código con `#[derive(Serialize, Deserialize)]`), `clap` (análisis declarativo de argumentos CLI), `tokio` y `axum` (servidores HTTP/gRPC asíncronos con extractores con seguridad de tipos), `sqlx` (consultas SQL verificadas contra el esquema de la base de datos en tiempo de compilación), `tracing` (telemetría estructurada con *spans* jerárquicos conscientes de tareas asíncronas), `proptest` (pruebas basadas en propiedades con reducción automática de contraejemplos) y `criterion` (microbenchmarking estadístico). Al diseñar APIs públicas, las *Rust API Guidelines* recomiendan aceptar parámetros genéricos mediante `impl AsRef<Path>` o `&str` en lugar de `&String` o `&Vec<T>`, implementar `From`/`TryFrom` y garantizar que las *feature flags* de Cargo sean estrictamente aditivas para que la unión de características solicitadas por distintos crates en el árbol de dependencias nunca provoque incompatibilidades de compilación.

8.2 Bloque H: Evaluación integral y los 4 proyectos progresivos (incluyendo rust-kv-engine verificado)

El Bloque H articula cuatro proyectos progresivos obligatorios: (1) una herramienta CLI de auditoría e indexación de ficheros con sumas criptográficas y salida estructurada; (2) un analizador sintáctico de

copia cero (*zero-copy*) para registros de configuración y telemetría; (3) un microservicio HTTP concurrente con limitación de tasa (*token bucket*) y apagado ordenado (*graceful shutdown*); y (4) el motor de almacenamiento Key-Value persistente `projects/rust-kv-engine`. En este cuarto proyecto, cada mutación `SET` o `DEL` se empaqueta en una trama binaria `[crc32: 4 B][op: 1 B][key_len: 4 B][val_len: 4 B][key][val]` de 13 B de cabecera fija y se sincroniza en el fichero de registro de escritura anticipada (*Write-Ahead Log*, WAL) antes de actualizar el índice en memoria protegido por `Arc<RwLock<HashMap<String, Vec<u8>>>>`:

```
pub fn encode_wal_frame_size(key: &str, value: &[u8]) -> usize {
    // 4B CRC32 + 1B opcode + 4B key_len + 4B val_len + key + value
    13 + key.len() + value.len()
}
```

Durante la recuperación tras un fallo eléctrico o caída del proceso, el motor recorre secuencialmente el WAL, recalcula el polinomio IEEE 802.3 CRC32 (`0xEDB88320`) de cada trama y rechaza de forma determinista cualquier registro truncado o corrompido en disco.

Problema resuelto

Enunciado En el motor `rust-kv-engine`, cada trama del Write-Ahead Log (WAL) tiene una cabecera fija de 13 B (4 B de CRC32, 1 B de código de operación, 4 B de longitud de clave y 4 B de longitud de valor). Si se registran $N = 2000$ operaciones `SET` donde cada clave mide $L_k = 16$ B y cada valor mide $L_v = 99$ B, calcula el tamaño exacto en bytes del fichero WAL resultante antes de ejecutar `compact()`.

Datos

- $N = 2000$ — Número de operaciones SET registradas
- $H = 13\,\mathrm{B}$ — Cabecera binaria fija por trama WAL
- $L_{kv}^{L} = 16\,\mathrm{B}, 99\mathrm{B}$ — Longitudes de clave y valor

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Tamaño total del fichero Write-Ahead Log (WAL) para N tramas

$$B_{\mathrm{WAL}} = 2000 \cdot (13 + 16 + 99)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos 13 + 16 + 99 = 128 B por trama y multiplicamos por 2000 tramas

$$B_{\mathrm{WAL}} = 2000 \cdot (13 + 16 + 99)$$

$$(2000 \cdot ((13 + 16) + 99))$$

Resultado

$$Resultado = 256000. El fichero WAL ocupa exactamente 256000 bytes (250 KiB) en disco.$$

Interpretación El fichero WAL ocupa exactamente 256 000 bytes (250 KiB) en disco.

Comprobación de los pasos

- Paso 1: 256000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0053] (Básico, Numérico) Calcula el tamaño en bytes de una trama WAL de `rust-kv-engine` con cabecera de 13 B, clave de 11 B y valor de 40 B ($13 + 11 + 40$).
2 [PRB-0054] (Básico, Desarrollo por pasos) Un fichero WAL contiene 5000 registros sobre un conjunto de solo 800 claves únicas (cada una actualizada varias veces). Si `compact()` conserva únicamente el último valor de cada clave viva (64 B por trama), ¿cuántos bytes ocupa el WAL compactado?
3 [PRB-0055] (Intermedio, Desarrollo por pasos) En el problema anterior, ¿cuántos bytes de espacio en disco libera la operación `compact()` respecto a las 5000 tramas originales de 64 B?
4 [PRB-0056] (Intermedio, Diseño) Un workspace de Cargo contiene 6 crates de biblioteca y cada crate define 12 pruebas unitarias y 3 pruebas de integración. ¿Cuántas pruebas ejecuta en total `cargo test -workspace`?

Apéndice A

Notación

Símbolo	Significado	Unidades
$T : \text{Send} + \text{Sync}$	Cotas de rasgo de transferencia y compartición segura entre hilos	adimensional
'a	Parámetro genérico de tiempo de vida (lifetime) de una referencia	adimensional
alignof(T)	Alineamiento de dirección de memoria requerido por el tipo T	B
sizeof(T)	Tamaño estático en bytes de un tipo Sized en pila	B

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 A. Introducción, historia, filosofía y modelo de compilación de Rust

Problema resuelto [PRB-0001]
En una arquitectura de 64 bits ($W = 8$ B), un vector en Rust `Vec<T>` se representa en la pila mediante tres campos de tamaño palabra (`ptr`, `len`, `cap`), mientras que una rebanada `&[T]` es un puntero ancho de dos palabras (`ptr`, `len`). Calcula el tamaño total en pila en bytes de una estructura que contiene dos `Vec<u8>` y tres rebanadas `&[u8]`.
Respuesta

$Resultado = 96. La estructura ocupa exactamente 96 bytes en la pila (48 B de los dos vectores y 48 B de las tres rebanadas), indeper$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0003]
Verificación ejecutable con `rustc -test` del tamaño exacto en bytes de `&u64`, `&[u64]` y `Vec<u64>` mediante `std::mem::size_of`.
Ejercicio 1 [PRB-0004]
Calcula el tamaño en bytes en la pila de un `Vec<f64>` en una arquitectura de 64 bits (3×8).
Respuesta 24
Ejercicio 2 [PRB-0005]
Calcula el tamaño en bytes de un puntero ancho `&dyn std::fmt::Display` en una CPU de 64 bits (2×8).
Respuesta 16
Ejercicio 3 [PRB-0006]
Si un binario contiene 45 funciones genéricas instanciadas cada una para 4 tipos concretos distintos mediante monomorfización, ¿cuántas funciones especializadas se emiten antes de la eliminación de código muerto?
Respuesta 180
Ejercicio 4 [PRB-0007]
Un búfer de 4096 elementos `u32` (4 bytes por elemento) está referenciado por un `Vec<u32>` (24 bytes en pila). ¿Cuántos bytes totales suma la cabecera en pila más la región en el montículo?
Respuesta 16408

Cálculos reproducidos

$((2 \cdot (3 \cdot 8)) + (3 \cdot (2 \cdot 8))) = 96$

C.2 B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

Problema resuelto [PRB-0008]
Una cadena en Rust contiene 10 caracteres ASCII (1 B cada uno en UTF-8) y 5 caracteres acentuados españoles (`á`, `é`, `í`, `ó`, `ú`, que ocupan 2 B cada uno en UTF-8). Calcula la diferencia en bytes entre representarla como `&str` en UTF-8 frente a un array `[char; 15]` de valores escalares Unicode (4 B por `char`).
Respuesta

$Resultado = 40. La representación UTF-8 nativa de Rust ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String y \&$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0010]
Verificación ejecutable con `rustc -test` de aritmética saturada, verificada y longitud UTF-8 frente a conteo de `chars()`.
Ejercicio 1 [PRB-0011]
Calcula el tamaño en bytes de un array `[u64; 32]` alojado en la pila (32×8).
Respuesta 256
Ejercicio 2 [PRB-0012]
Una cadena UTF-8 contiene 8 caracteres ASCII (1 B), 4 letras griegas (2 B) y 2 emojis (4 B). ¿Cuál es el valor de `s.len()` en bytes?
Respuesta 24
Ejercicio 3 [PRB-0013]
Calcula el resultado de `250u8.wrapping_add(15)` en aritmética modular de 8 bits ($250 + 15 - 256$).
Respuesta 9
Ejercicio 4 [PRB-0014]
Si `Vec::with_capacity(128)` reserva espacio para 128 elementos `i32` (4 B cada uno) e insertamos 50 elementos, ¿cuántos bytes de capacidad reservada permanecen aún libres en el montículo?
Respuesta 312

Cálculos reproducidos

$((15 \cdot 4) - ((10 \cdot 1) + (5 \cdot 2))) = 40$

C.3 D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

Problema resuelto [PRB-0015]
Un vector `Vec<u64>` contiene $N = 10000$ enteros de 8 bytes en el montículo en una máquina de 64 bits ($W = 8$ B). Calcula cuántos bytes menos se copian al pasar el vector por transferencia de propiedad (*move*, que solo copia las 3 palabras de la cabecera en pila) frente a invocar `.clone()` (que copia la cabecera más los 10000×8 bytes del montículo).
Respuesta

$$Resultado = 80000. La transferencia de propiedad (move) ahorra la copia de 80000 bytes en el montículo y se ejecuta en tiempo co$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0017]
Verificación ejecutable con `rustc -test` de préstamos con tiempos de vida explícitos `'a`, rebanadas mutables disjuntas (`split_at_mut`) y Non-Lexical Lifetimes.
Ejercicio 1 [PRB-0018]
Calcula cuántos bytes en la pila se copian al mover un `String` en una arquitectura de 64 bits (3×8).
Respuesta 24
Ejercicio 2 [PRB-0019]
Si dividimos una rebanada de 1024 elementos mediante `split_at_mut(300)`, ¿cuántos elementos contiene la segunda sub-rebanada mutable?
Respuesta 724
Ejercicio 3 [PRB-0020]
Una estructura contiene 4 campos `u64` que implementan `Copy`. ¿Cuántos bytes se copian en cada asignación por valor (4×8)?
Respuesta 32
Ejercicio 4 [PRB-0021]
Un analizador léxico de copia cero (*zero-copy*) genera 500 tokens `&'a str` (16 B cada uno) sobre un texto fuente de 32 000 bytes sin clonar subcadenas. ¿Cuántos bytes ocupan en total los 500 descriptores de token?
Respuesta 8000

Cálculos reproducidos

$(((3 \cdot 8) + (10000 \cdot 8)) - (3 \cdot 8)) = 80000$

C.4 D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

Problema resuelto [PRB-0022]
Una tabla hash abierta almacena $N = 65536$ ranuras de tipo `Option<std::num::NonZeroU64>` en una arquitectura de 64 bits ($W = 8$ B). Gracias a la optimización de nicho, cada `Option<NonZeroU64>` ocupa exactamente 8 bytes en lugar de 16 bytes (8 bytes de dato más 8 bytes de discriminante alineado). Calcula cuántos kilobytes (dividiendo entre 1024) se ahorran en la tabla completa gracias a la optimización de nicho.
Respuesta

$Resultado = 512. La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entra$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0024]
Verificación ejecutable con `rustc -test` de la optimización de nicho (`size_of::<Option<&u64>>() == 8`) y propagación de errores con `?`.
Ejercicio 1 [PRB-0025]
En una arquitectura de 64 bits, ¿cuántos bytes ocupa `Option<Box<u64>>` gracias a la optimización de nicho?
Respuesta 8
Ejercicio 2 [PRB-0026]
¿Cuántos bytes ocupa en total un array `[Option<NonZeroU32>; 1000]` sabiendo que cada elemento ocupa 4 bytes?
Respuesta 4000
Ejercicio 3 [PRB-0027]
Un `enum` sin nicho tiene una variante de 24 bytes con alineamiento de 8 bytes y una etiqueta de 1 byte. Tras añadir 7 bytes de relleno (*padding*) para mantener el alineamiento a múltiplo de 8, ¿cuál es su tamaño total en bytes?
Respuesta 32
Ejercicio 4 [PRB-0028]
Una línea de caché de CPU mide 64 bytes. ¿Cuántos elementos `Option<NonZeroU64>` (8 B cada uno) caben en una única línea de caché?
Respuesta 8

Cálculos reproducidos

$\frac{(65536 \cdot (16-8))}{1024} = 512$

C.5 E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

Problema resuelto [PRB-0029]
En una arquitectura de 64 bits ($W = 8$ B), se aloja un bloque `Rc< [u8; 1024] >` ($S_T = 1024$ B) y se crean $k = 5$ manejadores `Rc::clone` en distintas estructuras en la pila. Calcula la memoria total consumida (los 5 punteros en la pila más el único bloque `RcBox` en el montículo con sus 2 contadores `usize` y los 1024 bytes de datos).
Respuesta

$Resultado = 1080. El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente a$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0031]
Verificación ejecutable con `rustc -test` de tuberías de iteradores, `Rc/Weak` y mutabilidad interior con `RefCell`.
Ejercicio 1 [PRB-0032]
Calcula la suma de los cuadrados de los números pares del 2 al 10 ($\$2^2 + 4^2 + 6^2 + 8^2 + 10^2\$$) evaluada por `(1..=10).filter(|x| x% 2 == 0).map(|x| x * x).sum()`.
Respuesta 220
Ejercicio 2 [PRB-0033]
¿Cuántos bytes de metadatos añaden los dos contadores (`strong` y `weak`) de `RcBox<T>` en una arquitectura de 64 bits (2×8)?
Respuesta 16
Ejercicio 3 [PRB-0034]
Si se crean 1 `Rc` original, 4 clones fuertes (`Rc::clone`) y 3 referencias débiles (`Rc::downgrade`), y luego se destruyen 2 clones fuertes, ¿cuál es el valor actual de `Rc::strong_count`?
Respuesta 3
Ejercicio 4 [PRB-0035]
Un flujo de datos procesa 5000 enteros mediante `iter().filter(...).map(...).fold(...)` sin llamar a `collect()`. ¿Cuántos vectores dinámicos intermedios se alojan en el montículo durante el recorrido?
Respuesta 0

Cálculos reproducidos

$((5 \cdot 8) + ((2 \cdot 8) + 1024)) = 1080$

C.6 E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

Problema resuelto [PRB-0036]
Un servidor de mensajería mantiene $C = 50000$ conexiones TCP simultáneas. Si cada máquina de estados `Future` en Tokio ocupa 256 B (0,25 KiB) mientras que una pila mínima de hilo cuesta 64 KiB, calcula la memoria total en MiB (dividiendo los KiB entre 1024) que consumen las 50 000 tareas asíncronas.
Respuesta

$Resultado = 12{,}20703125. Las 50000 conexiones asíncronas consumen tan solo 12{,}20703125 MiB de RAM, frente a los 3125 Mi$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0038]
Verificación ejecutable con `rustc -test` de concurrencia paralela con `std::thread::scope`, `Arc<Mutex<T»` y canales `mpsc`.
Ejercicio 1 [PRB-0039]
Cuatro hilos incrementan un `AtomicU64` exactamente 25 000 veces cada uno con `fetch_add(1, Ordering::Relaxed)`. ¿Cuál es el valor final exacto del contador?
Respuesta 100000
Ejercicio 2 [PRB-0040]
Un vector de 10 000 elementos se divide equitativamente entre 8 hilos mediante `chunks(1250)` dentro de `std::thread::scope`. ¿Cuántos elementos procesa cada hilo?
Respuesta 1250
Ejercicio 3 [PRB-0041]
Una `async fn` tiene 3 variables locales `u64` (8 B cada una) vivas simultáneamente en su punto `.await` más grande, más 8 bytes de discriminante de estado. ¿Cuántos bytes ocupa esa máquina de estados `Future`?
Respuesta 32
Ejercicio 4 [PRB-0042]

Un canal acotado `mpsc::sync_channel(64)` almacena mensajes de 128 bytes cada uno. ¿Cuántos bytes como máximo ocupan los mensajes en tránsito cuando el búfer está lleno?
Respuesta 8192

Cálculos reproducidos

$\frac{(50000 \cdot 256)}{(1024 \cdot 1024)} = 12{,}20703125$

C.7 F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

Problema resuelto [PRB-0043]
Una estructura declarada con `#[repr(C)]` contiene en este orden exacto: `a: u8` (1 B), `b: u64` (8 B, requiere alineamiento a múltiplo de 8), `c: u8` (1 B) y `d: u32` (4 B, requiere alineamiento a múltiplo de 4), con alineamiento total de la estructura a múltiplo de 8 bytes. Calcula la diferencia en bytes para un array de $N = 1000$ estructuras entre `#[repr(C)]` (24 B cada una) y el empaquetado óptimo de `repr(Rust)` que ordena `b: u64`, `d: u32`, `a: u8`, `c: u8` ($8 + 4 + 1 + 1 + 2$ padding $= 16$ B).
Respuesta

$$Resultado = 8000. La reordenación automática de campos de repr(Rust) ahorra 8000 bytes (un 33{,}3\,\% de reducción de huella enc$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0045]
Verificación ejecutable con `rustc -test` de `safe_split_at_mut`, `macro_rules!` y diferencia real de tamaño entre `#[repr(C)]` y `repr(Rust)`.
Ejercicio 1 [PRB-0046]
Calcula el tamaño en bytes de `BadLayoutC { a: u8, b: u64, c: u8 }` bajo `#[repr(C)]` en 64 bits ($1 + 7 + 8 + 1 + 7$$).
Respuesta 24
Ejercicio 2 [PRB-0047]
Calcula el tamaño en bytes de `GoodLayoutRust { a: u8, b: u64, c: u8 }` reordenado por el compilador de Rust ($8 + 1 + 1 + 6$$).
Respuesta 16
Ejercicio 3 [PRB-0048]
Un registro SIMD de 256 bits (`AVX2`) procesa carriles `f32` de 32 bits en paralelo. ¿Cuántos números `f32` opera en una sola instrucción?
Respuesta 8
Ejercicio 4 [PRB-0049]
En un microcontrolador ARM Cortex-M `#![no_std]` con 64 KiB de SRAM (65536 B), el firmware reserva 8192 bytes para la pila y 4096 bytes para variables estáticas. ¿Cuántos bytes quedan disponibles para buffers estáticos de DMA?
Respuesta 53248

Cálculos reproducidos

$(1000 \cdot (24 - 16)) = 8000$

C.8 G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

Problema resuelto [PRB-0050]
En el motor `rust-kv-engine`, cada trama del Write-Ahead Log (WAL) tiene una cabecera fija de 13 B (4 B de CRC32, 1 B de código de operación, 4 B de longitud de clave y 4 B de longitud de valor). Si

se registran $N = 2000$ operaciones `SET` donde cada clave mide $L_k = 16$ B y cada valor mide $L_v = 99$ B, calcula el tamaño exacto en bytes del fichero WAL resultante antes de ejecutar `compact()`.
Respuesta

$$Resultado = 256000. El fichero WAL ocupa exactamente 256000 bytes (250 KiB) en disco.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0052]
Verificación ejecutable con `rustc -test` del algoritmo CRC32 IEEE 802.3 y empaquetado de trama WAL empleado en `projects/rust-kv-engine`.
Ejercicio 1 [PRB-0053]
Calcula el tamaño en bytes de una trama WAL de `rust-kv-engine` con cabecera de 13 B, clave de 11 B y valor de 40 B ($13 + 11 + 40$).
Respuesta 64
Ejercicio 2 [PRB-0054]
Un fichero WAL contiene 5000 registros sobre un conjunto de solo 800 claves únicas (cada una actualizada varias veces). Si `compact()` conserva únicamente el último valor de cada clave viva (64 B por trama), ¿cuántos bytes ocupa el WAL compactado?
Respuesta 51200
Ejercicio 3 [PRB-0055]
En el problema anterior, ¿cuántos bytes de espacio en disco libera la operación `compact()` respecto a las 5000 tramas originales de 64 B?
Respuesta 268800
Ejercicio 4 [PRB-0056]
Un workspace de Cargo contiene 6 crates de biblioteca y cada crate define 12 pruebas unitarias y 3 pruebas de integración. ¿Cuántas pruebas ejecuta en total `cargo test -workspace`?
Respuesta 90

Cálculos reproducidos

$(2000 \cdot ((13 + 16) + 99)) = 256000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 30 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios. Ecuaciones: 1 reproducida numéricamente.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-rust-nomicon (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-introduccion-historia-modelo-compilacion-rust scientific
REV-0002: cap02-entorno-toolchain-cargo-nivel-principiante scientific
REV-0003: cap03-ownership-borrowing-lifetimes-modelo-memoria scientific
REV-0004: cap04-tipos-algebraicos-traits-genericos-errores scientific
REV-0005: cap05-closures-iteradores-smart-pointers-mutabilidad-interior scientific
REV-0006: cap06-concurrencia-sin-miedo-send-sync-async-tokio-pin scientific
REV-0007: cap07-nivel-experto-unsafe-ffi-macros-wasm-embedded scientific
REV-0008: cap08-ecosistema-herramientas-evaluacion-proyectos-rust scientific

Apéndice E

Plan de estudio

Bloques A−C: Fundamentos, Entorno y Nivel Principiante en Rust

✓A. Introducción, historia, filosofía y modelo de compilación de Rust

- Comprender el origen histórico de Rust (Graydon Hoare, Mozilla Research, Rust Foundation) y las ediciones 2015, 2018, 2021 y 2024.
- Trazar la tubería de compilación desde el código fuente hasta código máquina nativo pasando por HIR, THIR, MIR y LLVM IR.
- Contrastar las abstracciones de coste cero y la seguridad espacial y temporal de memoria sin recolector de basura frente a C, C++ y Go.

2 conceptos planificados.

✓B−C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

- Configurar y gestionar toolchains con rustup, perfiles de compilación en Cargo.toml, resolución determinista con Cargo.lock, rustfmt y clippy.
- Dominar el modelo de vinculación inmutable por defecto (let vs let mut), sombreado (shadowing), tipos escalares y compuestos, y control de flujo expresivo.
- Distinguir la representación UTF-8 de String y &str frente a arrays [T; N], tuplas y vectores Vec<T>.

2 conceptos planificados.

Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad

✓D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

- Enunciar y aplicar las tres leyes del sistema de propiedad (ownership) y diferenciar semántica Move, Copy y Clone.
- Aplicar la ley de exclusión mutua de referencias (XOR entre alias y mutabilidad: múltiples &T o una única &mut T).
- Anotar explícitamente tiempos de vida genéricos ('a, 'static), comprender las tres reglas de elisión y el comprobador NLL.

2 conceptos planificados.

✓D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

- Modelar invariantes de dominio mediante tipos producto (struct) y tipos suma (enum) haciendo irrepresentables los estados inválidos.
- Diseñar jerarquías de polimorfismo paramétrico y ad-hoc con rasgos (traits), tipos asociados, cotas where y coherencia (regla del huérfano).

- Gestionar errores recuperables e irrecuperables mediante Option<T>, Result<T, E>, el operador de propagación ‘¿ y conversiones From/Into.

2 conceptos planificados.

Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe

✓E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

- Distinguir la representación y captura de entorno en clausuras según la jerarquía de rasgos FnOnce, FnMut y Fn.
- Diseñar tuberías perezosas de iteradores de coste cero y demostrar su equivalencia en ensamblador frente a bucles manuales.
- Seleccionar con rigor entre Box<T>, Rc<T>, Arc<T>, Cell<T>, RefCell<T>, Cow<’a, B> y Weak<T> evitando fugas por ciclos de referencias.

2 conceptos planificados.

✓E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

- Demostrar cómo los rasgos automáticos Send y Sync garantizan en tiempo de compilación la ausencia de carreras de datos.
- Combinar hilos con ámbito (std::thread::scope), canales MPSC, Arc<Mutex<T», Arc<RwLock<T» y tipos atómicos con órdenes de memoria.
- Comprender la compilación de funciones async a máquinas de estados sin pila (stackless coroutines), el rasgo Future::poll y la garantía estructural de Pin<P>.

2 conceptos planificados.

✓F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

- Enumerar exactamente los cinco superpoderes de un bloque unsafe y encapsularlos tras interfaces seguras con contratos Safety documentados.
- Construir puentes FFI bidireccionales con C (#[repr(C)], extern "C", #[no_mangle]) y auditar ausencias de Undefined Behavior con Miri.
- Escribir macros declarativas (macro_rules!) e higiénicas, comprender macros procedimentales y compilar crates para WebAssembly y sistemas empotrados (#![no_std]).

2 conceptos planificados.

Bloques G–H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos

✓G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

- Dominar el ecosistema estándar de producción en Rust: serde, clap, tokio, axum, sqlx, tracing, thiserror, anyhow, proptest y criterion.

- Diseñar arquitecturas modulares con Cargo Workspaces, feature flags aditivas, semántica SemVer e idioms (AsRef, Borrow, FromStr, Builder).
- Construir y verificar los 4 proyectos integradores progresivos en Rust, incluyendo el motor Key-Value concurrente con WAL y CRC32.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-rust-nomicon. https://doc.rust-lang.org/reference/src-rust-nomicon.html

[27] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[28] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

[29] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html

[30] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html